

Preparazione Orale

PlayConnect - Connected Games Platform

Objectif: savoir expliquer, demontrer et defendre le projet en discussion individuelle.

REST	MQTT	Edge/offline
Microservizi	Docker	MySQL
Demo	Codice	Domande piege

Preparazione orale - PlayConnect

Obiettivo: arrivare alla discussione sapendo spiegare il progetto, avviarlo, navigarlo, difendere le scelte tecniche e rispondere alle domande individuali.

0. Cosa valuta il professore

Dalla lezione introduttiva:

- la parte laboratorio e separata dalla teoria;
- la discussione riguarda documentazione, codice scritto e verifica del funzionamento;
- la valutazione è individuale anche se il progetto è di gruppo;
- i temi di laboratorio sono microservizi, API REST, protocolli IoT, MQTT, autenticazione, progettazione di software distribuito, database, frontend e backend.

Quindi non basta dire "il progetto funziona". Devi saper spiegare:

- che problema risolve;
- quali requisiti copre;
- quali componenti ci sono;
- come passa un evento dal sensore al database e al display;
- perché sono stati scelti REST, MQTT, microservizi, Docker e MySQL;
- quali limiti didattici esistono e come li miglioreresti in produzione.

1. Spiegazione da bambino di 5 anni

Immagina un calciobalilla in un bar.

Quando qualcuno fa goal, un sensore se ne accorge. Il sensore parla con una piccola scatola vicino al gioco, chiamata edge. L'edge manda un messaggio a Internet. Un server riceve il messaggio, aggiorna il punteggio e dice al display vicino al gioco cosa mostrare.

Se Internet non funziona, l'edge non butta via il goal. Lo mette in una lista di attesa. Quando Internet torna, invia i goal rimasti in coda.

PlayConnect fa questo per più giochi, più locali, più giocatori e anche tornei.

2. Discorso iniziale da 2 minuti

PlayConnect è una piattaforma per collegare giochi fisici tradizionali, come calciobalilla o freccette, a una piattaforma centrale. L'obiettivo è registrare partite, punteggi, statistiche e tornei usando sensori, dispositivi edge e comunicazione di rete.

Il sistema ha quattro ruoli: amministratore piattaforma, amministratore locale, amministratore gioco e giocatore. L'amministratore gioco definisce tipi di gioco e sensori; l'amministratore locale installa giochi, edge, sensori e attuatori; il giocatore consulta partite e statistiche; la piattaforma gestisce locali, utenti e tornei.

Architetturalmente abbiamo separato frontend, API Gateway, tre microservizi backend, broker MQTT Mosquitto, MySQL ed edge service. REST viene usato per le operazioni classiche di gestione e consultazione. MQTT viene usato per gli eventi dei sensori e i comandi agli attuatori, perché è leggero, asincrono e adatto a dispositivi IoT.

Il flusso principale è: il sensore genera un evento, l'edge lo pubblica su un topic MQTT, il Match Service lo riceve, valida l'evento, aggiorna partita e database, poi pubblica un comando verso gli attuatori. Se l'edge è offline, salva gli eventi in una coda JSON e li sincronizza quando torna online.

3. Mappa mentale del sistema

Componenti:

- Frontend: pagine HTML/CSS/JS per i quattro ruoli.
- API Gateway: unico ingresso REST su `localhost:3000/api`.
- Catalog Service: utenti, locali, giochi, tipi di gioco, sensori, edge e attuatori.
- Match Service: partite, eventi MQTT, punteggi, deduplicazione, attuatori.
- Tournament Service: squadre, tornei, calendario, statistiche e classifiche.
- Mosquitto: broker MQTT.
- Edge Service: simulatore del dispositivo fisico locale.
- MySQL: database persistente.
- Docker Compose: avvia tutto in modo ripetibile.

Frase chiave: REST serve per "chiedere e modificare dati"; MQTT serve per "mandare eventi e comandi in modo asincrono".

4. Strumenti usati

- Node.js: runtime JavaScript lato server.
- Express: framework per creare API REST.
- mqtt: libreria Node per collegarsi al broker MQTT.
- mysql2: driver Node per parlare con MySQL.
- MySQL 8: database relazionale.
- Mosquitto: broker MQTT.
- Docker e Docker Compose: container e orchestrazione locale.
- Nginx: serve i file statici del frontend.
- OpenAPI/Swagger YAML: documentazione API.
- Node test runner: test automatici backend.
- ReportLab/python-docx: generazione della relazione finale PDF/DOCX.

5. Demo da saper fare senza panico

Avvio:

```
docker compose down -v
docker compose up --build -d
```

Controllo:

```
docker compose ps
```

Aprire:

- applicazione: `http://localhost:8080`
- edge locale: `http://localhost:8090`
- health API: `http://localhost:3000/api/health`

Account:

- platform / platform123
- localadmin / local123
- gameadmin / game123

- client / client123
- mario / mario123
- luigi / luigi123

Percorso demo consigliato:

1. Apri health API e mostra servizi online.
2. Login gameadmin, mostra tipi di gioco e modelli sensore.
3. Login localadmin, mostra giochi, edge, sensori e attuatori.
4. Avvia una partita Mario-Luigi.
5. Apri live match e premi simulazione MQTT.
6. Mostra punteggio che cambia.
7. Apri edge locale e mostra stato, ultimo evento e attuatori.
8. Metti edge offline, invia evento, mostra coda.
9. Rimetti online, mostra coda svuotata.
10. Login platform, mostra tornei, utenti e statistiche.

Test:

```
cd backend
npm test
npm run check
cd ..
node scripts/validate-project.js
node scripts/integration-test.js
```

Risultati attesi:

- 27 test backend superati;
- 127 controlli progetto superati;
- 7 test integrazione superati.

6. Domande probabili e risposte

Perche MQTT e non solo REST?

REST funziona bene quando un client chiede una risorsa e riceve una risposta. Per i sensori IoT e meglio MQTT perche il dispositivo pubblica eventi piccoli su un topic, il broker li distribuisce, il sistema e piu disaccoppiato e gestisce meglio riconnessione e comunicazione asincrona.

Perche REST e ancora presente?

Perche REST e comodo per gestire utenti, locali, giochi, partite, tornei e statistiche dal browser. MQTT non sostituisce tutto: nel progetto REST e per gestione dati, MQTT per eventi e attuatori.

Cos'e un broker MQTT?

E un intermediario. Chi produce messaggi pubblica su un topic; chi e interessato si iscrive al topic. Publisher e subscriber non devono conoscersi direttamente.

Cosa sono i topic?

Sono indirizzi logici dei messaggi. Esempio: locales/1/games/2/matches/5/events. Significa: eventi della partita 5 del gioco 2 nel locale 1.

Perche QoS 1?

QoS 1 significa consegna almeno una volta. E utile se non vogliamo perdere eventi. Siccome puo arrivare un duplicato, usiamo event_uuid per evitare di contare due volte lo stesso evento.

Come funziona l'offline?

Se l'edge non può pubblicare su MQTT, salva l'evento in `offline-queue.json`. Quando torna online, legge la coda e ripubblica gli eventi. Il server usa UUID per non duplicare.

Cosa fa il Match Service quando riceve un goal?

Valida il payload, controlla che partita/gioco/locale corrispondano, controlla duplicati, aggiorna il punteggio in MySQL, salva l'evento e aggiorna gli attuatori.

Perché microservizi?

Per separare responsabilità: catalogo, partite e tornei cambiano per motivi diversi. Il gateway nasconde questa divisione al frontend.

Qual è il limite dei microservizi nel vostro progetto?

Il database è condiviso. È una scelta didattica per semplificare demo e installazione. In produzione ogni servizio dovrebbe possedere meglio i propri dati o avere confini più rigidi.

Perché Node.js se il laboratorio mostra Java/Spark/JDBC?

Il corso ammette anche altri framework. Node/Express implementa gli stessi concetti: API REST, concorrenza asincrona, accesso DB tramite driver, MQTT, microservizi. La scelta aiuta per eventi e I/O non bloccante.

Come gestite l'autenticazione?

C'è login con password hash PBKDF2. Dopo il login, per la demo didattica le API usano X-User-Id. In produzione useremmo JWT o sessioni firmate, HTTPS e gestione sicura dei segreti.

Perché non avete usato Keycloak?

Keycloak/OAuth2 è stato studiato come evoluzione produttiva. Per questo progetto didattico abbiamo scelto una autenticazione più semplice per concentrarci su requisiti principali: REST, MQTT, edge, offline, microservizi e demo funzionante.

Dove sono i test più importanti?

In `backend/tests` per regole e servizi. `scripts/integration-test.js` verifica anche il flusso end-to-end con Docker, MQTT, offline e attuatore.

7. Programma di 7 giorni

Giorno 1 - Capire il progetto come storia

Obiettivo: saper spiegare PlayConnect senza codice.

Da studiare:

- README.md
- START_HERE.md
- docs/00-relazione-finale.md, sezioni 1-4
- docs/01-specifiche-funzionali.pdf
- docs/02-architettura.pdf

Esercizio:

- ripeti il discorso da 2 minuti tre volte;
- disegna su carta: frontend, gateway, servizi, MQTT, edge, database;
- spiega la differenza tra REST e MQTT con parole tue.

Giorno 2 - Avvio e demo

Obiettivo: saper avviare il progetto e fare la demo senza leggere.

Da fare:

- esegui `docker compose up --build -d`;
- apri frontend, edge e health;
- prova tutti gli account;
- fai una partita e una simulazione MQTT;
- prova offline e sync.

Esercizio:

- cronometrati: devi fare una demo completa in 10-12 minuti;
- prepara una frase per ogni schermata.

Giorno 3 - Architettura e code walkthrough

Obiettivo: sapere dove sono le cose nel codice.

Da studiare:

- `docker-compose.yml`
- `gateway/server.js`
- `backend/serviceServer.js`
- `backend/mqttClient.js`
- `simulator/simulator.js`
- `backend/services/matchEventService.js`
- `backend/services/actuatorService.js`

Esercizio:

- segui un goal nel codice: edge -> MQTT -> Match Service -> DB -> attuatore.
- prepara la risposta: "cosa succede se arriva due volte lo stesso evento?"

Giorno 4 - Database, ruoli e requisiti

Obiettivo: collegare requisiti, ruoli e tabelle.

Da studiare:

- `database/init.sql`
- `backend/middleware/auth.js`
- `backend/utils/accessRules.js`
- controller principali in `backend/controllers`
- `docs/16-matrice-requisiti.pdf`

Esercizio:

- per ogni ruolo, cita tre azioni consentite;
- spiega perché un giocatore non può amministrare un locale;
- spiega cosa contiene una partita e cosa contiene un torneo.

Giorno 5 - Corsi di laboratorio collegati al progetto

Obiettivo: sapere rispondere alle domande teoriche del lab.

Da ripassare in laboratorio:

- REST HTTP Servers: risorse, URI, GET/POST/PUT/DELETE, JSON.
- REST multithreading: richieste concorrenti e approccio event-driven.
- Publish/Subscribe: publisher, subscriber, broker, topic.
- Mosquitto: broker MQTT, publish/subscribe, QoS.
- IoT e Arduino MQTT: sensori, attuatori, edge.
- Microservices architecture style: separazione servizi, vantaggi, limiti.
- OAuth2 e Keycloak: sapere dire perché e una possibile evoluzione.
- JDBC: concetto di driver DB, anche se noi usiamo mysql2.

Esercizio:

- per ogni corso, scrivi una riga: "nel nostro progetto questo si vede in..."

Giorno 6 - Questions pièges

Obiettivo: non farti destabilizzare.

Allenati su queste questions:

- Perché non un monolite?
- Perché un database condiviso se dite microservizi?
- Cosa succede se MQTT consegna due volte?
- Cosa succede se l'edge resta offline a lungo?
- Come proteggeresti davvero le API?
- Perché non Keycloak?
- Perché Node invece di Java?
- Dove sono i limiti del progetto?
- Quale parte hai implementato o sai spiegare meglio?
- Come testeresti un bug sul punteggio?

Regola:

- prima rispondi semplice;
- poi collega al codice;
- poi cita un limite/miglioramento.

Giorno 7 - Simulazione orale completa

Obiettivo: provare come se fosse l'esame.

Routine:

1. Presentazione progetto in 2 minuti.
2. Demo completa in 10-12 minuti.
3. Spiegazione architettura in 5 minuti.
4. Walkthrough di un evento MQTT.
5. Domande su REST/MQTT/microservizi/offline.

6. Domande individuali sul codice.

7. Chiusura: limiti e miglioramenti futuri.

Ultima checklist:

- Docker parte?
- Health API online?
- Login funzionano?
- Simulazione MQTT funzionante?
- Offline queue funzionante?
- Test passano?
- Sai spiegare ogni file principale?
- Sai ammettere i limiti senza sembrare impreparato?

8. Risposte da imparare quasi a memoria

Il cuore del progetto

Il cuore del progetto è la trasformazione di eventi fisici in eventi digitali affidabili. L'edge raccoglie l'evento, MQTT lo porta al server, il Match Service aggiorna stato e punteggio, e REST permette agli utenti di consultare e gestire il sistema.

La scelta più importante

La scelta più importante è separare REST e MQTT: REST per operazioni amministrative e consultazione, MQTT per eventi IoT asincroni. Questo rende il sistema più coerente con sensori, edge e offline.

Il limite principale

Il limite principale è che, per scopi didattici, il sistema usa un database condiviso e autenticazione semplificata dopo il login. In produzione introdurrei JWT/Keycloak, HTTPS, segreti esterni e confini dati più rigidi tra microservizi.

Perché il progetto è valido

Il progetto copre i requisiti richiesti: quattro ruoli, giochi configurabili, sensori, attuatori, edge offline, MQTT, REST, microservizi, tornei multi-locale, statistiche, documentazione UML/OpenAPI e test automatici.